

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..16

CONFIGURE HTTP CALLS

 Copy as Markdown

Configure HTTP calls

Two situations call for customizing how your app sends requests: a web app that must include cookies on cross-origin calls, and apps that want the platform-native network stacks on iOS and Android. Both are handled by the `httpClientOverride` parameter on the generated `Client`, which swaps the underlying HTTP transport. The `serverUrl` in the examples below is the resolved server address from [calling endpoints](#).

Include CORS credentials on web

By default, browser requests do not include cookies or HTTP authentication credentials in cross-origin requests. If your app relies on cookie-based sessions or similar mechanisms, pass a `BrowserClient` with `withCredentials` enabled:

```
import 'package:http/browser_client.dart';

final client = Client(
  serverUrl,
  httpClientOverride: BrowserClient()..withCredentials = true,
);
```

Allow credentials on the server

On the server, Serverpod adds CORS headers to API responses by default through `httpResponseHeaders` and `httpOptionsResponseHeaders` on the `Serverpod` constructor. The defaults allow cross-origin `POST` requests from any origin (`Access-Control-Allow-Origin: *`) and permit common request headers such as `Authorization` on preflight `OPTIONS` requests.

Credential-aware requests require `Access-Control-Allow-Credentials: true` and a specific origin instead of the wildcard. Override the defaults in your `lib/server.dart` (or wherever you construct `Serverpod`):



```
import 'src/generated/serverpod.dart';

/// The starting point of the Serverpod server.
void run(List<String> args) async {
  final pod = Serverpod(
    args,
    httpResponseHeaders: Headers.build((mh) {
      mh.accessControlAllowOrigin = AccessControlAllowOriginHeader.origin(
        origin: Uri.parse('http://localhost:49660'), // Your Flutter web app origin
      );
      mh.accessControlAllowCredentials = true;
      mh.accessControlAllowMethods = AccessControlAllowMethodsHeader.methods(
        [Method.post],
      );
    }),
    httpOptionsResponseHeaders: Headers.build((mh) {
      mh.accessControlAllowHeaders = AccessControlAllowHeadersHeader.headers([
        'Content-Type',
        'Authorization',
        'Accept',
        'User-Agent',
        'X-Requested-With',
      ]);
    }),
  );

  // Start the server
  await pod.start();
}
```

Set `origin` to the exact origin of your Flutter web app (scheme, host, and port). In production, list each allowed origin explicitly.

Use platform-native HTTP clients

You can also override the default HTTP client with a platform-native HTTP client. On iOS and macOS, you can use `cupertino_http` to route traffic through `NSURLSession`. On Android, you can use `cronet_http` to use the Cronet network stack.

Add the corresponding package to your Flutter app's `pubspec.yaml` before using these clients.

```
import 'dart:io';

import 'package:cronet_http/cronet_http.dart';
import 'package:cupertino_http/cupertino_http.dart';
import 'package:http/http.dart' as http;

import 'package:my_project_client/my_project_client.dart';

void main() async {
  http.Client? httpClient;
```

```

if (Platform.isAndroid) {
  final engine = CronetEngine.build(
    cacheMode: CacheMode.memory,
    cacheMaxSize: 2 * 1024 * 1024,
    userAgent: 'my-app');
  httpClient = CronetClient.fromCronetEngine(engine, closeEngine: true);
} else if (Platform.isIOS || Platform.isMacOS) {
  final config = URLSessionConfiguration.ephemeralSessionConfiguration()
    ..cache = URLCache.withCapacity(memoryCapacity: 2 * 1024 * 1024)
    ..httpAdditionalHeaders = {'User-Agent': 'my-app'};
  httpClient = CupertinoClient.fromSessionConfiguration(config);
}

final client = Client(
  serverUrl,
  httpClientOverride: httpClient,
);
}

```

Support web with conditional imports

The above example does not work if your app also targets web, since `dart:io` is not available there. Put the platform-specific `http.Client` creation logic behind a conditional import instead:

```

import 'src/http_client_stub.dart'
  if (dart.library.io) 'src/http_client_io.dart';

final client = Client(
  serverUrl,
  httpClientOverride: createHttpClient(),
);

```

The stub file provides a fallback for platforms without `dart:io` (such as web), where the default client is used:

src/http_client_stub.dart

```

import 'package:http/http.dart' as http;

http.Client? createHttpClient() ⇒ null;

```

The `dart:io` implementation wraps the platform-native client from the example above in a top-level `createHttpClient()` function:

src/http_client_io.dart

```

import 'dart:io';

```

```
// Same imports as the platform-native example above.
```

```
http.Client? createHttpClient() {  
  if (Platform.isAndroid) {  
    // ... return the CronetClient shown above  
  } else if (Platform.isIOS || Platform.isMacOS) {  
    // ... return the CupertinoClient shown above  
  }  
  return null;  
}
```